

TravelMate Rental Services

System Architecture & Component Registry (10 Key Modules)

● ● ● Frontend / src / pages / AIVerification.jsx

```
import React, { useRef, useState, useEffect } from 'react';
import * as cocoSsd from '@tensorflow-models/coco-ssd';
import * as tf from '@tensorflow/tfjs';

export default function AIVerification() {
  const [model, setModel] = useState(null);
  useEffect(() => {
    async function loadResources() {
      await tf.ready();
      const loadedModel = await cocoSsd.load({ base: 'lite_mobilenet_v2' });
      setModel(loadedModel);
    }
    loadResources();
  }, []);
  // Real-time vehicle detection logic ...
}
```



```
}, []);  
// Real-time vehicle detection logic ...  
}
```

Backend / com.TravelMate.entities / User.java

```
@Entity(name="user")  
@Table(name="users")  
@Getter @Setter @AllArgsConstructor @NoArgsConstructor @Builder  
public class User {  
    @Id private String userId;  
    @Column(name="User_name") private String username;  
    @Column(unique = true, nullable = false) private String email;  
    private String password;  
    private boolean isActive=false;  
    @Enumerated(EnumType.STRING) private loginProvider loginBy=loginProvider.SELF;  
    @ElementCollection(fetch = FetchType.EAGER)  
    @CollectionTable(name = "user_roles",joinColumns = @JoinColumn(name = "userId"))  
    Set roles=new HashSet<>();  
}
```

Frontend / src / App.jsx

● ● ● Frontend / src / App.jsx

```
function App() {  
  return (  
  
    } />  
    } />  
    } />  
    } />  
    } />  
  
  );  
}
```

● ● ● Backend / com.TravelMate / TravelMateApplication.java

```
package com.TravelMate;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;
```



```
);  
}
```

● ● ● Backend / com.TravelMate / TravelMateApplication.java

```
package com.TravelMate;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class TravelMateApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(TravelMateApplication.class, args);  
    }  
}
```

● ● ● Frontend / src / pages / Signin.jsx

```
const handleSubmit = async (e) => {  
    try {  
        const formData = new FormData(e.target);  
        const data = {  
            email: formData.get('email'),  
            password: formData.get('password')  
        };  
        const response = await axios.post(`${API_URL}/api/auth/signin`, data);  
        if (response.status === 200) {  
            // Success  
            // Redirect to home page  
            // ...  
        }  
    } catch (error) {  
        // Error  
        // ...  
    }  
}
```



```
@SpringBootApplication
public class TravelMateApplication {
    public static void main(String[] args) {
        SpringApplication.run(TravelMateApplication.class, args);
    }
}
```

● ● ● Frontend / src / pages / Signin.jsx

```
const handleSubmit = async (e) => {
    try {
        const response = await fetch("http://localhost:8081/api/users/login", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ email, password }),
        });
        const data = await response.text();
        if (response.ok) {
            localStorage.setItem("token", data);
            navigate("/");
        }
    } catch (err) { setError("Network error."); }
};
```



```
    } catch (err) { setError("Network error."); }  
};
```

Backend / com.TravelMate.controllers / pageController.java

```
@RestController  
@CrossOrigin(origins = "http://localhost:5173")  
@RequestMapping("/api/users")  
public class pageController {  
    @Autowired private UserRepository userRepository;  
    @Autowired private JwtUtil jwtUtil;  
  
    @PostMapping("/login")  
    public ResponseEntity loginUser(@RequestBody UserDetails user) {  
        Optional existingUser = userRepository.findByEmail(user.getEmail());  
        if (existingUser.isPresent() && encoder.matches(user.getPassword(), existingUser.get().getPassword())) {  
            String token = jwtUtil.generateToken(user.getEmail());  
            return ResponseEntity.ok().body("{\"token\": \"" + token + "\"}");  
        }  
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid credentials");  
    }  
}
```



```
}  
}
```

● ● ● Frontend / src / pages / Checkout.jsx

```
const Checkout = () => (  
  
  Checkout  
  Payment and Final Review.  
  
);
```

● ● ● Frontend / src / pages / BookingHistory.jsx

```
const BookingHistory = () => {  
  useEffect(() => {  
    const token = localStorage.getItem("token");  
    const allBookings = JSON.parse(localStorage.getItem("vm_bookings") || "[]");  
    const userBookings = allBookings.filter(b => b.userToken === token);  
    setBookings(userBookings);  
  }, []);  
}
```



```
);
```

● ● ● Frontend / src / pages / BookingHistory.jsx

```
const BookingHistory = () => {  
  useEffect(() => {  
    const token = localStorage.getItem("token");  
    const allBookings = JSON.parse(localStorage.getItem("vm_bookings") || "[]");  
    const userBookings = allBookings.filter(b => b.userToken === token);  
    setBookings(userBookings);  
  }, []);  
  // Rendering historical rental data ...  
}
```

● ● ● Backend / com.TravelMate.entities / Role.java

```
package com.TravelMate.entities;  
  
public enum Role {  
    uploader,  
    seeker  
}
```


Backend / com.TravelMate.entities / Role.java

```
package com.TravelMate.entities;

public enum Role {
    uploader,
    seeker
}
```

Backend / pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
  </dependency>
</dependencies>
```



```
package com.TravelMate.entities;
```

```
public enum Role {  
    uploader,  
    seeker  
}
```

● ● ● Backend / pom.xml

```
<dependencies>  
    <dependency>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-starter-data-jpa</artifactId>  
    </dependency>  
    <dependency>  
        <groupId>io.jsonwebtoken</groupId>  
        <artifactId>jjwt-api</artifactId>  
        <version>0.11.5</version>  
    </dependency>  
</dependencies>
```